

ACI · AIMLCZG557 · COMPANION READER

Intelligent Agents & Environments

PEAS, environment properties, agent architectures, problem formulation

Session 2 · Read alongside the lecture slides · Every slide example worked out in full

How to use this companion. The slides give you the headlines; this reader gives you the *why*. Every concept is explained in plain English, and every slide exercise is worked out step by step. Colour-coded boxes guide you. **Blue** = the core idea in one breath. **Amber** = a real-life analogy before any math. **Green** = a problem solved with every step shown. **Red** = a common trap to avoid. **Purple** = the one line to remember.

1 What this session covers

CS#2 covers four tightly linked ideas:

1. **PEAS** — how to describe any AI agent's task in four parts.
2. **Environment types** — six pairs of properties that describe the world an agent lives in.
3. **Agent architectures** — five designs, from the simplest reflex rule to a full learning system.
4. **Problem formulation** — how to turn a goal into a search problem with five components.

The session also previews uninformed and informed **search strategies** (BFS, DFS, UCS, A*), but the algorithm detail is in CS#3. This companion covers everything up to that preview.

2 PEAS: describing any AI task

Before we build an agent, we must be clear about what the agent is trying to do and what tools it has. **PEAS** is the four-part checklist: **P**erformance measure, **E**nvironment, **A**ctuators, **S**ensors.

The intuition

Think of hiring a new employee. Before day one you write the job description. That description has four parts: the score (Performance), the workplace (Environment), the tools they can use (Actuators), and the information they can see (Sensors). PEAS is that job description for an AI agent.

The four components, defined.

Performance measure

The score we use to judge the agent. It must be defined *before* building the agent. Write it in terms of world outcomes, not internal steps.

Environment	Everything in the world that the agent interacts with — the objects, the people, the physical setting.
Actuators	The outputs the agent has — all the ways it can change the world (displays, wheels, arms, text responses).
Sensors	The inputs the agent receives — all the ways it can observe the world (cameras, keyboards, microphones, joint-angle sensors).

★ Everyday picture

A taxi driver: Performance = safe arrival, profits, legal compliance. Environment = roads, pedestrians, other cars, customers. Actuators = steering wheel, accelerator, brakes, horn. Sensors = cameras, GPS, speedometer, microphone. Every self-driving car project starts with exactly this checklist.

2.1 Worked PEAS examples from the slides

The lecture walks through two PEAS examples. Let us work each one fully.

Worked example: PEAS for a Medical Diagnosis System

The task: an AI system that interviews a patient and suggests a diagnosis.

Step 1. Performance measure. Healthy patient outcome; low cost; no lawsuits. We measure success by whether patients recover and costs stay down — not by internal algorithm steps.

Step 2. Environment. The patient, the hospital setting, and the medical staff around the system. These are all the entities whose states matter.

Step 3. Actuators. A screen display — the agent's only output is text: questions to ask, test orders, diagnoses, treatments, referrals.

Step 4. Sensors. A keyboard. Everything the agent learns comes from what a human types: symptoms, test results, patient answers.

Sense-check: actuators and sensors are *physical* links between the agent and the world. A screen cannot move a robot arm; a keyboard cannot hear a heartbeat. The PEAS forces us to be precise about these boundaries.

Worked example: PEAS for a Part-Picking Robot

The task: a robot arm that picks parts off a conveyor belt and drops them in the correct bins.

Step 1. Performance measure. Percentage of parts sorted into the correct bins. Higher percentage = better agent.

Step 2. Environment. The conveyor belt, the parts moving on it, and the target bins. The physical factory floor is the environment.

Step 3. Actuators. A jointed robotic arm and a gripping hand. These are the only ways the agent can change the world.

Step 4. Sensors. A camera (to see part position and type) and joint-angle sensors (to know the arm's current pose). Without these, the arm is blind.

Sense-check: both agents above operate in very different environments, yet the same four-part checklist describes them completely.

Worked example: PEAS exercise: Interactive English Tutor (e.g. Grammarly)

This is Slide Exercise 1. Work it yourself first, then compare.

Step 1. Performance measure. Grammar accuracy of the user's writing; speed of feedback; user satisfaction; number of repeated errors (lower is better). You could also include % of suggestions accepted.

Step 2. Environment. The text document the user is writing; the web or app interface; the user (who types and reads suggestions).

Step 3. Actuators. A screen display: underlines, pop-up suggestions, rewrite proposals, explanations in plain English.

Step 4. Sensors. Keyboard (character-by-character text input); clipboard (pasted text); possibly the document context (title, style guide chosen).

There is no single right answer for a PEAS — the goal is to be specific and consistent. If you said “performance = user happiness”, that is too vague to measure. Be concrete.

Where this shows up in ML. Every ML system is an agent. Consider a language model. Performance = helpfulness and accuracy. Environment = the conversation and user. Actuators = text output (tokens). Sensors = the input prompt (tokens). Defining PEAS up front stops teams from optimising the wrong metric — a common failure mode in production ML.

✓ Key takeaway

PEAS = Performance, Environment, Actuators, Sensors. Always define performance in terms of world outcomes, not internal behaviour. Actuators are outputs; sensors are inputs.

3 Environment types: six dimensions

Not all environments are the same. Two agents can have identical PEAS but face very different challenges depending on how their environments behave. The lecture gives six binary dimensions for classifying environments.

The intuition

Think of environments on six light switches. Each switch is on or off. Flipping a switch changes how hard the agent's problem is. A fully-observable, deterministic, static, episodic, discrete, single-agent environment is the easiest world possible. The real world has every switch in the hard position.

3.1 Dimension 1: Observability

Fully observable: the agent's sensors give complete access to the current state of the environment at every moment. **Partially observable:** the sensors miss something — too noisy, limited range, hidden information. **Non-observable:** the agent can see nothing; it must act blind.

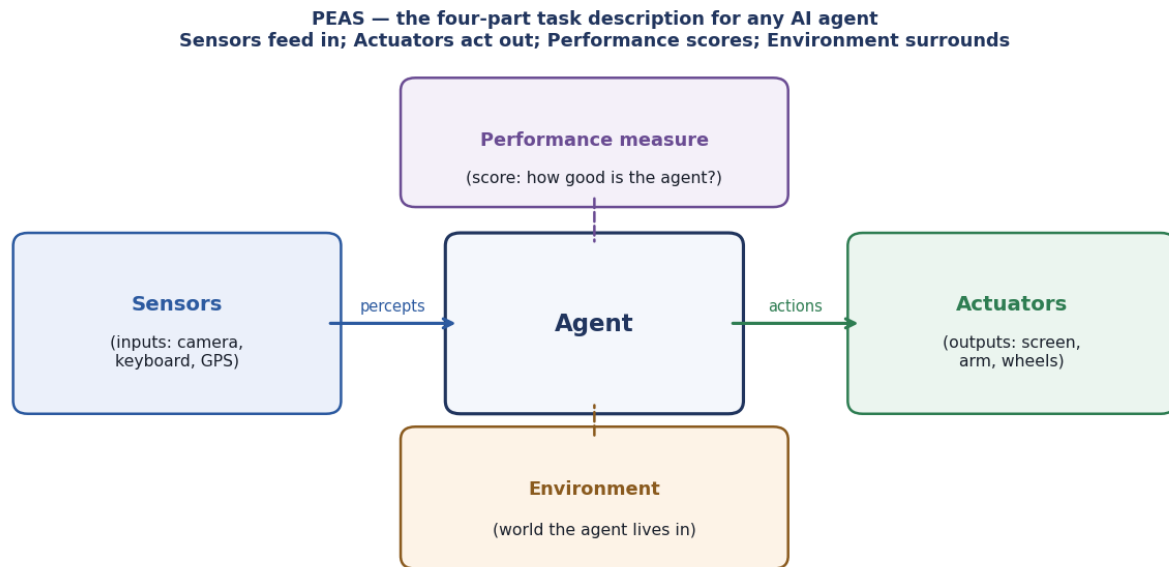


Figure 1: PEAS as a diagram. Sensors carry percepts into the agent (left arrow). Actuators carry actions out to the world (right arrow). The performance measure scores the agent from outside. The environment is everything the agent interacts with. This diagram applies to every AI agent, from a spam filter to a self-driving car.

★ **Everyday picture**

Chess (fully observable): both players see the whole board. Poker (partially observable): you see your own cards but not your opponents'. A robot in a dark room (non-observable): it cannot see a thing and must rely on touch or prior maps.

3.2 Dimension 2: Determinism

Deterministic: the same action from the same state always produces the same next state. There is no randomness. Example: a calculator. If you press $2 + 2 =$, you always get 4.

Stochastic: the next state has an element of chance. The same action can produce different results. Examples: weather, robot hitting a football goal, autonomous driving (tyres slip, other drivers surprise you).

A special case: if an environment is deterministic except for the actions of *other agents*, it is called **strategic**.

★ **Everyday picture**

Filling in a crossword puzzle: deterministic. The rules are fixed and the puzzle does not change while you think. Driving in traffic: stochastic. Even if you steer perfectly, a pedestrian might step out unexpectedly.

3.3 Dimension 3: Episodic vs. Sequential

Episodic: the agent's experience is cut into independent, atomic "episodes." Each episode is a single percept followed by a single action. The current episode does not depend on what happened in earlier episodes. Examples: classifying whether an image contains a cat; a spam filter deciding on one email.

Sequential: the current action affects future situations. The agent must plan ahead. Examples:

chess (one move changes all future possibilities); an investment portfolio bot (today's trade affects tomorrow's prices).

✗ Watch out

“Episodic” does not mean short. A single image classification is episodic even if the model is large. “Sequential” does not mean long. Even a two-step decision tree is sequential if step 1 changes what options are available at step 2. The key question is: does this action affect the *choices* available later?

3.4 Dimension 4: Static vs. Dynamic

Static: the environment does not change while the agent is thinking. Examples: crossword puzzle, Rubik's cube. The tiles wait patiently.

Dynamic: the environment can change while the agent is deciding. Examples: driving (cars move), weather (rain starts), financial markets (prices tick).

Semidynamic: the environment itself does not change, but the agent's *performance score* changes with time. Example: chess with a clock. The board is static, but thinking too long costs you the game.

3.5 Dimension 5: Discrete vs. Continuous

Discrete: a finite, countable set of distinct states and actions. Example: chess has a large but countable number of board positions. A card game has a finite deck.

Continuous: states and actions take real-valued numbers with infinite precision. Examples: robot arm angles (any value from 0 to 360 degrees), taxi speed, GPS coordinates.

3.6 Dimension 6: Single agent vs. Multi-agent

Single agent: the agent is the only decision-maker. Any other objects in the environment behave predictably (they are part of the environment, not agents). Example: solving a Sudoku puzzle alone.

Multi-agent: other agents are present. They can sense and act. They may cooperate (factory robots), compete (a chess opponent), or simply pursue their own goals (other drivers on the road).

★ Everyday picture

Two chess players — multi-agent, competitive. Assembly line robots — multi-agent, cooperative. You solving a jigsaw puzzle alone — single agent.

3.7 The full environment table

The lecture shows a summary table. Let us reproduce it and read each row.

Task environment	Observable	Agents	Deterministic	Episodic	Static	Discrete
Crossword puzzle	Fully	Single	Deterministic	Sequential	Static	Discrete
Chess (with clock)	Fully	Multi	Deterministic	Sequential	Semi	Discrete
Poker	Partially	Multi	Stochastic	Sequential	Static	Discrete
Backgammon	Fully	Multi	Stochastic	Sequential	Static	Discrete
Taxi driving	Partially	Multi	Stochastic	Sequential	Dynamic	Continuous
Medical diagnosis	Partially	Single	Stochastic	Sequential	Dynamic	Continuous
Image analysis	Fully	Single	Deterministic	Episodic	Semi	Continuous
Part-picking robot	Partially	Single	Stochastic	Episodic	Dynamic	Continuous
Refinery controller	Partially	Single	Stochastic	Sequential	Dynamic	Continuous
English tutor	Partially	Multi	Stochastic	Sequential	Dynamic	Discrete

Key insight. The real world scores hard on all six dimensions. Simplifying any one dimension makes the problem much easier. That is why researchers start with toy environments (fully observable, discrete, single-agent) before tackling the real world.

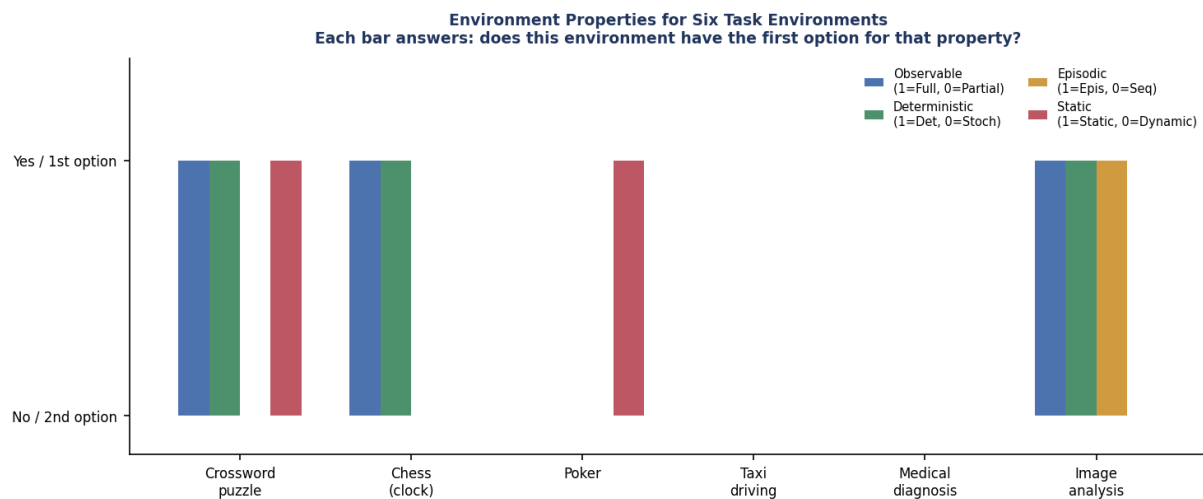


Figure 2: Environment properties for six task environments. A bar at 1 means the “easy” property (e.g. fully observable, deterministic); a bar at 0 means the “hard” property. Notice how taxi driving and medical diagnosis sit low on almost every dimension — they are hard problems for exactly this reason.

Where this shows up in ML. Reinforcement learning is designed for sequential, stochastic, partially observable environments. A deep RL agent for a video game (fully observable, discrete, single-agent) is far simpler to train than one for autonomous driving. Knowing the environment type tells you which algorithm family to try first.

✓ Key takeaway

Six dimensions: observable / deterministic / episodic / static / discrete / single-agent. The real world scores hard on all six. The environment type shapes the agent design.

4 Agent architectures: five designs

An agent is fully specified by its **agent function**: a mapping from percept sequences to actions. We cannot store this table for any real problem — it grows exponentially. Instead, we design programs that compute the action. The lecture gives five architectures, each adding one new capability.

The intuition

Think of five help-desk employees, each more capable than the last. Employee 1 reads the current ticket and fires a scripted reply (Simple Reflex). Employee 2 also checks notes from past tickets (Model-Based). Employee 3 knows the goal and plans toward it (Goal-Based). Employee 4 scores several plans and picks the best (Utility-Based). Employee 5 gets better each week by learning from feedback (Learning).

Five Agent Architectures — each step adds one new capability

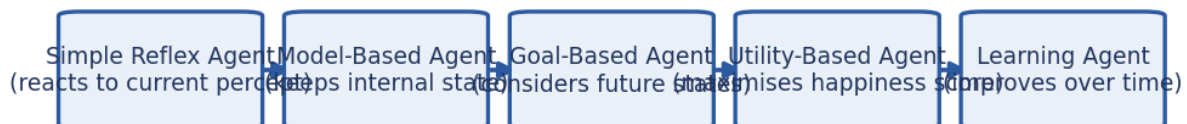


Figure 3: The five agent architectures as a capability ladder. Each step adds one new internal component. A learning agent at the top includes all the lower layers.

4.1 Simple Reflex Agent

A **simple reflex agent** looks only at the current percept. It matches the percept against a set of condition–action rules and fires the first rule that matches.

Algorithm sketch.

1. Interpret the raw percept into a description of the current world state.
2. Match the state against the rule set.
3. Return the action attached to the matched rule.

Vacuum-world example (from the slide). Two locations A and B; the vacuum can be at either; each location may be dirty or clean.

Condition	Action
Current location is dirty	Suck
Current location is A and clean	Move Right (to B)
Current location is B and clean	Move Left (to A)

Three rules cover every reachable situation. No memory of the past is needed.

⚠ Watch out

Simple reflex agents fail in partially observable environments. If the location sensor breaks, the agent cannot tell which room it is in. The rules then give the wrong action — or no action at all. Without a model of the world, one missing percept is fatal.

4.2 Model-Based Reflex Agent

A **model-based reflex agent** keeps an *internal state* that describes the parts of the world it cannot currently see. It uses two internal models to update that state:

Transition model

“How does the world evolve on its own?” (e.g. traffic lights change on a timer).

Sensor model

“How do my percepts relate to the true world state?” (e.g. my camera sometimes blurs fast-moving objects).

Every step: update the internal state using the transition model and the new percept, then apply condition–action rules to the updated state.

4.3 Goal-Based Agent

A **goal-based agent** goes further: it knows what state it is trying to reach (the *goal*). When choosing an action, it asks “will this action help me reach the goal?” This requires *search* and *planning* — reasoning about sequences of future actions.

Goal-based agents are more flexible than reflex agents. Changing the goal changes all behaviour automatically. A reflex agent needs a new rule set every time the task changes.

4.4 Utility-Based Agent

Goals are binary (reached or not). But some paths to the goal are better than others. A **utility-based agent** uses a **utility function** that maps states to a real number representing how “happy” the agent is in that state. It picks the action that leads to the highest expected utility.

★ Everyday picture

Two routes to the airport both get you there (both reach the goal). But one takes 20 minutes and the other takes 45. A goal-based agent picks either one; a utility-based agent picks the 20-minute route because it maximises the utility (comfort, speed, fuel cost weighed together).

4.5 Learning Agent

A **learning agent** improves its own performance by learning from experience. It has four components:

Performance element

The part that actually acts — any of the four agent types above.

Critic

Compares the agent’s behaviour against a *performance standard* and gives a feedback signal (positive or negative).

Learning element

Uses the critic’s feedback to update the performance element so it acts better next time.

Problem generator

Suggests exploratory actions that may be sub-optimal now but will teach the agent something new (exploration vs. exploitation).

Concrete example from the slide. An autonomous car sees a red light at 50 metres and applies the brake. The performance element applies a force of 17 kN. The critic measures the resulting stop distance and compares it to ideal. If the car stopped 2 metres past the line, the

critic sends -2 feedback. The learning element adjusts the rule. Next time, from the same distance and speed, the car applies 15 kN and stops on time.

Worked example: Learning agent braking improvement

We trace one learning step for the autonomous car.

- Step 1. Percept.** Camera sees red light; speed sensor reads 50 km/h; distance sensor reads 50 m.
- Step 2. Performance element acts.** Current rule: “brake at 17 kN from 50 m.” Car applies 17 kN.
- Step 3. Resulting state.** Car stops 2 m past the stop line.
- Step 4. Critic evaluates.** Performance standard says “stop before the line.” The car overshot by 2 m. Feedback signal = -2 .
- Step 5. Learning element updates.** The learning element lowers the braking force in the rule: new rule is “brake at 15 kN from 50 m.”
- Step 6. Next time.** Same percept (50 km/h, 50 m). Car applies 15 kN and stops exactly at the line. Critic sends 0 (no error).

This is the same feedback loop used in reinforcement learning: observe a state, take an action, receive a reward signal, update the policy.

Where this shows up in ML. Every neural network trained with gradient descent is a learning agent. The model is the performance element. The loss function is the critic. Backpropagation is the learning element. Data augmentation explores new actions. In reinforcement learning the reward signal is the critic and the policy is the performance element.

✓ Key takeaway

Five architectures: Simple Reflex (rules on current percept) → Model-Based (adds internal state) → Goal-Based (adds planning) → Utility-Based (adds optimisation) → Learning (adds self-improvement). Each step adds one internal component.

5 Problem formulation: the five components

A **problem-solving agent** works in three phases: **Formulate** (define the problem), **Search** (find a solution), **Execute** (carry out the actions). This section covers Formulate.

The intuition

Formulating a search problem is like setting up a board game before playing. You place pieces at their starting positions (initial state). You write down the legal moves (actions). You agree on what the board looks like after each move (transition model). You define how to win (goal test) and how to score paths (path cost). Once these five things are fixed, any search algorithm can take over.

A **state** captures all information about the world that matters for making decisions. A **state space** is the set of all states reachable from the initial state. The state space forms a graph: nodes are states, arcs are actions. A **path** is a sequence of states linked by actions. The solution is a path from the initial state to a goal state.

5.1 The five components

Initial state Where the agent starts. Written formally: $\text{In}(\text{Arad})$ means “the agent is in Arad.”

Possible actions

All legal actions from any given state. From $\text{In}(\text{Arad})$, the legal actions are $\{\text{Go}(\text{Sibiu}), \text{Go}(\text{Timisoara}), \text{Go}(\text{Zerind})\}$.

Transition model

The function $\text{RESULT}(s, a)$ that gives the state reached by doing action a in state s . $\text{RESULT}(\text{In}(\text{Arad}), \text{Go}(\text{Sibiu})) = \text{In}(\text{Sibiu})$.

Goal test Checks whether a state is a goal. $\text{IsGoal}(\text{In}(\text{Bucharest})) = \text{Yes}$. Sometimes there is one fixed goal; sometimes there is a set of goal states.

Path cost A function that assigns a numeric cost to each complete path. Typically the sum of step costs. $\text{cost}(\text{In}(\text{Arad}), \text{Go}(\text{Sibiu})) = 140 \text{ km}$.

A **solution** is any path from the initial state to a goal state. An **optimal solution** is the one with the lowest path cost.

Five Components of a Problem Formulation Every search problem is fully described by these five pieces

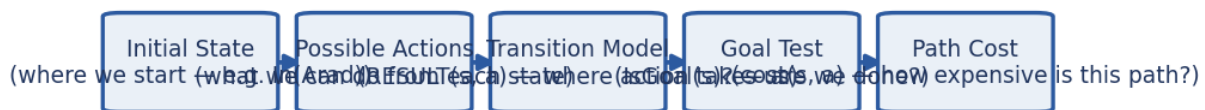


Figure 4: The five components of a problem formulation. Every search problem — from navigating a map to solving an 8-puzzle — is fully described by these five pieces. The arrows show the flow: initial state feeds into the action generator, which feeds the transition model, whose outputs are tested by the goal test. Path cost accumulates along every arc.

5.2 Worked example: Romania road map

The lecture uses the classic Romania map. Agent is in Arad; goal is Bucharest.

Worked example: Romania problem formulation

- Step 1. Initial state.** $\text{In}(\text{Arad})$.
- Step 2. Actions from Arad.** $\{\text{Go}(\text{Sibiu}), \text{Go}(\text{Timisoara}), \text{Go}(\text{Zerind})\}$. Three choices.
- Step 3. Transition model.** $\text{RESULT}(\text{In}(\text{Arad}), \text{Go}(\text{Sibiu})) = \text{In}(\text{Sibiu})$. Similarly for each other city.
- Step 4. Goal test.** $\text{IsGoal}(\text{In}(\text{Bucharest})) = \text{Yes}$. All other cities return No.
- Step 5. Path cost.** Sum of road distances.
- Step 5. One solution (not necessarily optimal).** $\text{Arad} \rightarrow \text{Sibiu} \rightarrow \text{Fagaras} \rightarrow \text{Bucharest}$. Total cost = $140 + 99 + 211 = 450 \text{ km}$.

Step 6. Sense-check. Is there a shorter path? Arad → Sibiu → Rimnicu Vilcea → Pitesti → Bucharest = 140 + 80 + 97 + 101 = 418 km. Search algorithms will find this if they minimise path cost.

The formulation is complete. We have not solved the problem yet; we have only written down what the problem *is*. The search algorithm comes next (CS#3).

5.3 Worked example: Vacuum World

A toy problem from the slide. Two locations (A and B); each may be dirty or clean; agent can be at either location. Three actions: Left, Right, Suck.

Worked example: Vacuum World problem formulation

Step 1. State count. 2 agent positions × 2 states for location A × 2 states for location B = 8 total states. These are all possible world configurations.

Step 2. Initial state. Any of the 8 states with the agent's position and each room's dirt status specified. Example: agent at A, A dirty, B dirty.

Step 3. Actions. Three: Left, Right, Suck.

Step 4. Transition model. Left moves agent to A (no effect if already at A). Right moves agent to B (no effect if already at B). Suck removes dirt from the current location (no effect if clean).

Step 5. Goal test. Both A and B are clean. Two of the 8 states are goal states.

Step 6. Path cost. Each action costs 1 step. Total path cost = total number of actions taken. Optimal solution minimises steps to get both rooms clean.

Starting from “agent at A, A dirty, B dirty”: optimal plan is Suck (A cleaned), Right, Suck (B cleaned) — 3 steps, cost 3.

Vacuum World: all 8 possible states (V = vacuum agent, red = dirty cell, green border = goal)

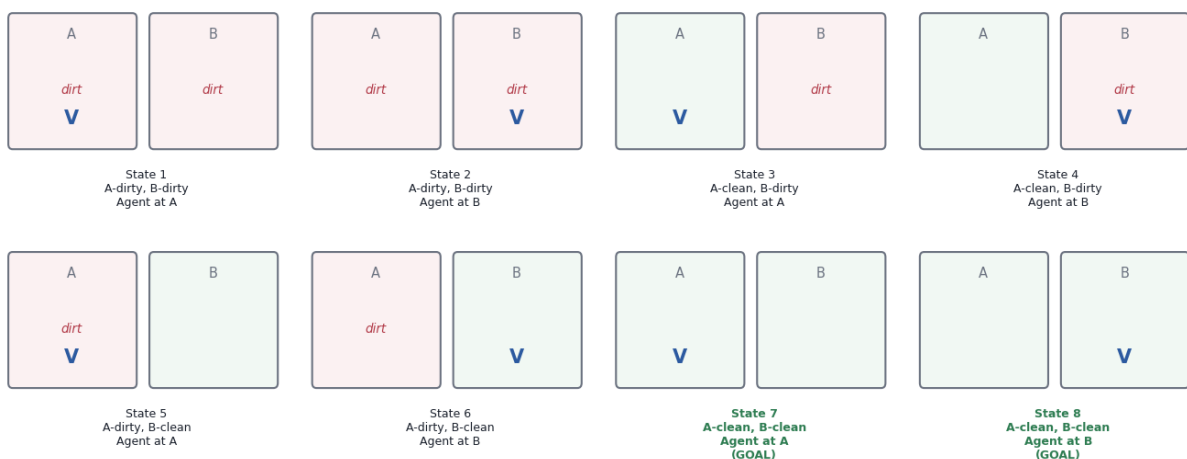


Figure 5: All 8 states of the Vacuum World. V = the vacuum agent; red cells are dirty; green-framed states are goals (both rooms clean). A search algorithm explores this graph to find the shortest path from any start state to a goal state.

5.4 Worked example: 8-Puzzle

Another classic from the slide. A 3×3 grid with tiles 1–8 and one blank. Slide tiles into the blank to reach the goal configuration.

Worked example: 8-Puzzle problem formulation

- Step 1. State count.** $9!$ permutations of 9 cells, but only half are reachable from any given start (the others are in a different parity class): so $9!/2 = 181\,440$ reachable states.
- Step 2. Initial state.** Any permutation of tiles 1–8 with one blank. The slide shows tiles $[7, 2, 4; 5, _, 6; 8, 3, 1]$ as one example start.
- Step 3. Actions.** Move the blank: **Left, Right, Up, Down**. (“Moving the blank left” is the same as sliding the tile to the left of the blank rightward into the blank’s old position.)
- Step 4. Transition model.** The resulting board after moving the blank. The blank swaps position with the adjacent tile in the chosen direction.
- Step 5. Goal test.** Board matches the goal configuration $[_, 1, 2; 3, 4, 5; 6, 7, 8]$.
- Step 6. Path cost.** Each move costs 1. Optimal solution = fewest moves.
- Sense-check: 181 440 states is large but manageable. BFS would work but is slow; informed search (A* with Manhattan-distance heuristic) is far faster.

Where this shows up in ML. Neural Architecture Search (NAS) is a search problem. The state space is the set of possible network designs. Actions are design choices (add a layer, change a kernel size). The goal test is “accuracy $\geq$ target”; path cost is training time. AlphaGo uses Monte Carlo Tree Search with a learned value function as the heuristic.

✓ Key takeaway

A problem formulation has five parts: Initial State, Actions, Transition Model, Goal Test, Path Cost. A solution is a path to a goal; an optimal solution minimises path cost. These five parts are everything a search algorithm needs.

6 Search strategies: a preview

CS#2 ends with a preview of search. CS#3 covers the algorithms in full. Here is the map so you know where things fit.

A **search strategy** is the rule for choosing which node to expand next. It is the only thing that changes between algorithms. Strategies are judged on:

- **Completeness** — does it always find a solution if one exists?
- **Optimality** — does it find the lowest-cost solution?
- **Time complexity** — how many nodes does it generate?
- **Space complexity** — how many nodes does it hold in memory?

Three parameters set the scale: b = branching factor (max children per node); d = depth of the shallowest goal; m = maximum depth of the tree.

The intuition

Uninformed strategies explore blind. They follow a fixed rule (always go shallowest, or always go deepest) and hope to hit the goal eventually. Informed strategies use a **heuristic** $h(n)$. A heuristic estimates how far node n is from the goal. It steers search toward promising areas first. The heuristic is the GPS; uninformed search is wandering without one.

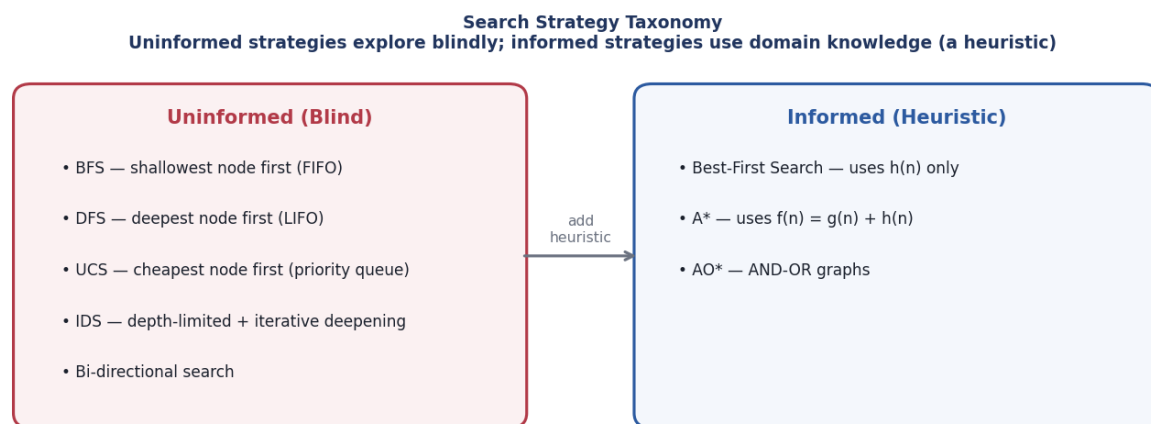


Figure 6: Two families of search strategies. Uninformed strategies (left, red) follow fixed structural rules with no knowledge of the goal's location. Informed strategies (right, blue) use a heuristic to prefer nodes that look closer to the goal. Moving from left to right adds one key ingredient: a domain knowledge function $h(n)$.

✓ Key takeaway

Uninformed search (BFS, DFS, UCS, IDS, Bi-directional) uses no domain knowledge. Informed search (Best-First, A*, AO*) uses a heuristic $h(n)$ to guide expansion. The algorithms and their complexity analysis are in CS#3.

Glossary & symbols

Key terms.

agent	Anything that perceives its environment through sensors and acts through actuators.
PEAS	The four-part task description: Performance, Environment, Actuators, Sensors.
percept	A single sensor reading — the raw input the agent gets at one moment.
actuator	A physical output channel — how the agent changes the world.
fully observable	The agent's sensors show the complete current state.
deterministic	The same action in the same state always gives the same next state.
episodic	Each decision is independent of all past decisions.
sequential	Current decisions affect future options; planning is needed.

static	The environment does not change while the agent is thinking.
dynamic	The environment can change while the agent is thinking.
semidynamic	The environment is static but the agent's score changes with time.
discrete	A finite, countable set of states and actions.
continuous	States or actions take real-valued (infinite) values.
simple reflex agent	Acts on condition–action rules applied to the current percept only.
model-based agent	Maintains an internal state to track unobservable parts of the world.
goal-based agent	Chooses actions by reasoning about whether they help reach a goal.
utility-based agent	Chooses actions to maximise a numeric happiness score.
learning agent	Improves its own performance by learning from feedback.
state	A complete description of the world at one moment.
state space	All states reachable from the initial state.
initial state	The state the agent starts in.
transition model	$\text{RESULT}(s, a)$ — the state reached by doing action a in state s .
goal test	A function that checks whether a state is a solution.
path cost	The total numeric cost of a path (sum of step costs).
heuristic	An estimate $h(n)$ of the cost from node n to the nearest goal.
uninformed search	A search strategy with no domain knowledge beyond the problem definition.
informed search	A search strategy that uses a heuristic to guide expansion.

Symbols at a glance.

Symbol	Meaning
b	branching factor — maximum number of children any node has in the search tree
d	depth of the shallowest goal node
m	maximum depth of the search tree (may be infinite)
$g(n)$	path cost from the initial state to node n (exact)
$h(n)$	heuristic estimate of the cost from n to the nearest goal
$f(n)$	$g(n) + h(n)$: estimated total cost of the cheapest path through n (used by A^*)
$\text{RESULT}(s, a)$	transition model: state reached from state s by action a
